SYSC 3020: Introduction to Software Engineering

Assignment 3: Design (Behaviour & Architecture) · Lab Walkthrough

Rinkesh Joshi · rinkeshjoshi@cmail.carleton.ca

SYSC 3020: Introduction to Software Engineering

Assignment 3: Design (Behaviour & Architecture) · Lab Walkthrough

Fall 2026

TA: Rinkesh Joshi · rinkeshjoshi@cmail.carleton.ca

Lab: LE1 · Wednesday, October 21, 2026 · 2:35 PM to 5:25 PM

Room: Mackenzie Building 4233

Assignment due: Friday, November 13, 2026 by 11:59 PM

Weight: 5% of final grade · graded out of 16

The Three Tasks at a Glance

Task A

Statecharts

Model ≥ 2 stateful entities from the code.

- States (from fields)
- Events (method calls)
- Guards [ ]
- Actions / …
- Won / lost = events, not states
- Debugger screenshot per chart

Output: ≥ 2 statecharts

- screenshots

Task B

Architecture

Recover the architecture of the whole system.

- Name the style + justify
- 4+1 views: logical · development · process · physical · +1 scenario (key press → model; UI refresh separately)

Output: Style + five views

Task C

ADRs & Trace

Record why it's built this way.

- ≥ 2 ADRs with code evidence + alternatives
- Extend the RTM in Requirement Yogi

Output: ADRs + Traceability section

Plus Sprint 3 running across all tasks in Jira (planning, stand-ups, board, review + retro).

> Same as A2: follow the separate Jira Guide slides for stand-ups and role labels. New: share the board with the instructor and the TAs (next slides).

What You Submit

Deliverables (one per team, on Brightspace)

1. architecture.pdf: exported from Confluence
   - ≥ 2 statecharts + debugger screenshots (Task A)
   - Architectural style + the 4+1 views (Task B)
   - ≥ 2 ADRs + Traceability section (Task C)
   - Team & process (roles, retro, AI Usage note)
2. uml-sources.zip: all your PlantUML `.puml` files
3. Jira board link: with the instructor and all the TAs added to your space, so the link opens

As in A2, list every team member (name + student ID) in the Brightspace submission.

Penalties to Watch

- Missing architecture.pdf → −3 marks
- Missing uml-sources.zip → −3 marks
- Jira link we cannot open → −3 marks
- Late → −15% of the full grade per day (accepted up to 48 h late)
- Submit only these three items

Presentation

A random sample of students may be asked to present; it must match what you submitted. Can't explain it → affects your individual grade.

Grading: 4 criteria × 4 pts = 16 total (State machines · Architecture recovery · Decision records & traceability · Process & reporting)

Grading Rubric: Quick View

State Machines (4 pts)

≥ 2 statecharts faithfully modelling the code: every state justified by a field / condition / event; any composite state clearly labelled as an abstraction; transitions verified with a debugger trace.

Architecture Recovery (4 pts)

Clear, correct architecture: style justified via packages / components, with all 4+1 views, including a convincing scenario.

Decision Records & Traceability (4 pts)

≥ 2 well-argued ADRs grounded in code evidence, with alternatives; RTM fully extended.

Process & Reporting (4 pts)

Jira board current with real sprint activity; AI usage disclosed; report complete and correctly formatted (unrendered diagrams cost marks).

Score Range	Meaning
14-16	Excellent work
10-13	Good work
6-9	Some improvement needed
0-5	Much improvement needed

Each criterion is scored 1-4 (see the rubric at the end of the handout).

From A2 to A3: What Changes

Reuse from Assignment 2

- CD-01: your class diagram tells you which classes hold state (Task A) and is the start of the logical view (Task B)
- SD-01 … SD-03: your traces of `Game.move` / `Level.move` are the start of the +1 scenario
- RTM in Yogi: you extend it, not restart it
- PlantUML, same Jira space, same accounts
- Penalties: 15%/day, −3 per missing item

New or Different

- Report is now architecture.pdf
- The Jira link is a penalised item (−3 if we can't open it); add the instructor too
- Scope: Task B covers the whole system, incl. `ui` and `sprite`
- Five 4+1 views, each with a diagram (physical: a short description is enough)
- New diagram types: state machine, package, (optional) deployment
- New writing: ADRs
- Debugger screenshots are required (we use IntelliJ)

> Note: You are analysing the design that exists, not changing the code and not guessing what the original developers historically considered.

Before You Draw › Lecture Notes

▒▒▒ Go through the posted slides first; use their vocabulary

Slides	Covers	Use it for
Behavioural Models	statecharts: states, events, guards, actions	Task A
Architectural Design	layered, pipe-and-filter, MVC	Task B: style
4+1 View Model (new, ATM)	five views; which diagram for which view	Task B: views
Modular Design	coupling / cohesion, SRP, OCP, DIP, ISP	ADR consequences

The handout calls the styles slides "the architecture lecture"; the 4+1 slides are a separate short deck.

▒▒▒ Not in the lectures

- ADRs → the handout's definition is all you need; Nygard's post shows the format (link 7)

Before You Draw › References & Links

Used in class / on the course page

1. Multi-layered app (UML model diagram): https://www.uml-diagrams.org/multi-layered-application-uml-model-diagram-example.html
2. MVC (MDN glossary): https://developer.mozilla.org/en-US/docs/Glossary/MVC
3. Architecture styles (Microsoft): https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/
4. SOLID in plain English: https://www.freecodecamp.org/news/solid-principles-explained-in-plain-english/

Extra references for A3

6. Kruchten, The "4+1" View Model (1995): https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf
7. Nygard, Documenting Architecture Decisions: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions
8. ATM state machine example: https://www.uml-diagrams.org/bank-atm-uml-state-machine-diagram-example.html
9. PlantUML state diagrams: https://plantuml.com/state-diagram
10. PlantUML component / package diagrams: https://plantuml.com/component-diagram
11. PlantUML activity diagrams (fork, swimlanes): https://plantuml.com/activity-diagram-beta
12. Swing painting (repaint() vs paint()): https://www.oracle.com/java/technologies/painting.html
13. IntelliJ IDEA download: https://www.jetbrains.com/idea/download/ · student licence: https://www.jetbrains.com/student/

Before You Draw › How to Use Those Links

Link	Use it for	Careful
1 Multi-layered	layers drawn as packages	its service / data layers don't exist here
2 MDN MVC	what M, V and C each do	web examples: map the roles, not the tech
3 Microsoft	a style = a set of constraints	its distributed styles are out of scope
4 SOLID	words for ADR consequences	Liskov is not in Modular Design; cite the link
8 ATM	composite states, entry / exit actions	—
12 Swing painting	`repaint()` vs `paint()`, which thread	needed for the UI refresh scenario

> Out of scope for A3: distributed styles from the Architectural Design slides (client/server, REST, SOA, microservices) and the distributed / cloud styles on link 3 (microservices, event-driven, big data, big compute; its N-tier is the layered idea across machines). The deployment is given: one JVM, one desktop app.

Sprint 3 › Run It in Jira

▒▒ Day 1: Sprint planning

- Once A2 is submitted: complete Sprint 2 (Board → Complete sprint), then plan Sprint 3
- Create the epic: "A3: Design: Behaviour & Architecture" (a new epic, same Jira space)
- Break it into tasks → assign each (shared task → sub-tasks; Jira Guide › Jira 4)
- One-line sprint goal → Create Sprint → Start Sprint
- Add the instructor and all the TAs to the space now (Jira Guide › Jira 10): kahani@sce.carleton.ca + the three TA emails. A link we can't open is −3 marks.

▒▒ Rotate roles (no one keeps a role two sprints running)

Member	Sprint 1 (A1)	Sprint 2 (A2)	Sprint 3 (A3)
A	Product Owner	Scrum Master	QA Lead
B	Scrum Master	QA Lead	Product Owner
C	QA Lead	Product Owner	Scrum Master

Update the role labels to match (Jira Guide › Jira 5).

▒▒ During the sprint + last day

- Stand-ups ≥ 2×/week as Jira comments: done / doing / blocked, each member from their own account
- Move tasks Backlog / To Do → In Progress → Done as you work; the board history is graded
- Last day: review + 3 bullets (keep / change / try) in the Team & process section

How You Work › Diagrams-as-Code (Again)

Same pipeline as A2

```
sm-01.puml ──render──▶ SM-01.png ──upload──▶ Confluence page ──export──▶ architecture.pdf
     │
     └──────────────────────── zip all .puml ──────────────────────▶ uml-sources.zip
```

Give every diagram and record an ID (suggested scheme)

ID	What	File
SM-01 … SM-03	Statecharts: Level, Player, Game (Task A)	sm-01.puml …
LV-01	Logical view (class or package diagram)	lv-01.puml
DV-01	Development view (package diagram)	dv-01.puml
PV-01	Process view	pv-01.puml
PHY-01	Physical view (a sentence, or a deployment diagram)	phy-01.puml (optional)
SC-01 · SC-02	+1 scenario: input flow · UI refresh	sc-01.puml · sc-02.puml
ADR-01, ADR-02	Decision records (text, no .puml)	on the page

The IDs are what your RTM rows and your screenshot captions point to.

> Graphviz: class, state, package / component and deployment diagrams need it to render (sequence and new-syntax activity diagrams, link 11, don't).
> Easiest: the PlantUML web server (https://www.plantuml.com/plantuml/uml/), nothing to install.
> Running PlantUML on your own laptop without Graphviz? Add the line !pragma layout smetana right after @startuml;
> PlantUML then does the layout itself. Setup steps: A2 slides.

Setup › IntelliJ IDEA (Our Debugger for Task A)

The debugger screenshots in Task A need an IDE. We use IntelliJ IDEA: since 2025.3 there is one download, and the former Community features are free (no licence, no sign-in needed).

Windows

- Download: https://www.jetbrains.com/idea/download/ → IntelliJ IDEA → Windows installer
- Run the installer, accept the defaults (tick Add "bin" folder to the PATH if asked)
- Or: JetBrains Toolbox app → install IntelliJ IDEA

macOS

- Same page → IntelliJ IDEA → macOS disk image (Apple Silicon or Intel: Apple menu → About This Mac)
- Open the `.dmg`, drag IntelliJ IDEA to Applications
- Or: `brew install --cask intellij-idea`

Student licence (optional)

https://www.jetbrains.com/student/ (Carleton email) unlocks the paid features. Nothing in A3 needs them: the debugger, Find Usages and Gradle support are in the free tier.

> Already have IntelliJ from another course? Skip the install; next slide still applies (JDK 11 + Gradle JVM).

Setup › Open the Repo in IntelliJ (JDK 11!)

Step 1: Open as a Gradle project

File → Open (or Open on the welcome screen) → select the repo folder (the one with build.gradle) → OK → Trust Project if asked → wait for the Gradle sync (progress bar at the bottom).

Step 2: If the sync fails, point IntelliJ at JDK 11

The repo's Gradle 5.3 only runs on Java 8-11; IntelliJ often picks a newer bundled JDK.

- Settings (Ctrl+Alt+S Windows / ⌘, macOS) → Build, Execution, Deployment → Build Tools → Gradle → Gradle JVM → your JDK 11 (A1 install)
- File → Project Structure (Ctrl+Alt+Shift+S / ⌘;) → Project → SDK → 11
- Gradle tool window (elephant icon, right sidebar) → Sync All Gradle Projects (circular arrows) → BUILD SUCCESSFUL in the Build pane

Step 3: Run the game from the IDE

Open src/main/java/nl/tudelft/jpacman/Launcher.java → green ▶ in the gutter next to main → Run 'Launcher.main()'. The game window opens; press Start. If it runs, the debugger will too.

What each step looks like: Appendix A (end of the deck).

> Hint: typo in a shortcut? Every action is also in the menus and in Help → Find Action (Ctrl+Shift+A / ⇧⌘A).

Setup › Debugger Basics (You'll Use All of These)

Action	Windows	macOS	Where you see it
Toggle a breakpoint	click the gutter · Ctrl+F8	click the gutter · ⌘F8	red dot on the line
Debug Launcher.main	▶ → Debug, then Shift+F9	▶ → Debug, then ^D	Debug window opens
Step Over (run this line)	F8	F8	Variables pane updates
Step Into (follow the call)	F7	F7	new frame on the stack
Resume (next breakpoint)	F9	⌘⌥R	
Find Usages	Alt+F7	⌥F7	Find tool window

The two panes your screenshots must show (Debug tool window)

- Frames (in the new UI: the left half of Threads & Variables): the call stack, top = current line, plus a thread dropdown (AWT-EventQueue-0 = Swing EDT; pool-N-thread-1 = an executor thread: the UI refresh or a ghost task)
- Variables (right half of the same tab): expand this → the field values (alive, inProgress, score, …) and the method parameters

Both panes, at the three Task A breakpoints: Appendix B (end of the deck).

> Default: a breakpoint suspends all threads (right-click the red dot → Suspend: All), so the ghosts freeze while you take the screenshot.

Task A › What You Actually Do

Step 1: Find the genuinely stateful entities

Look for fields that change at runtime and change behaviour (a method checks them before acting). The handout's candidates:

- Player: `alive` (`isAlive()`, `setAlive(…)`)
- Level: `inProgress` (`start()`, `stop()`, `isInProgress()`)
- Game: its own `inProgress` field (`start()`, `stop()`, `levelWon()`, `levelLost()`)

Step 2: For each one, read every method that touches the field

- Who sets it? (IntelliJ: right-click → Find Usages; `Alt+F7` Windows / `⌥F7` macOS)
- What does the method check first? What does it do besides setting the field?
- Is any value ever set back? (e.g., is `setAlive(true)` ever called?)

Step 3: Draw it, then cite the code beside it

Each state / transition / guard → class.method + line in the report.

Step 4: Verify it in the debugger

One screenshot per statechart: call stack + field value (slide Task A › Verify in the Debugger).

> Rule: don't invent states. "Paused", "Won", "Ready", "Respawning" are wrong unless a field or condition in the code backs them.

Task A › Reading a Statechart Out of the Code

In the code	In the statechart
a `boolean` / enum field	the states (one per value that matters)
the constructor's initial value	initial pseudostate ● → first state
the method that changes the field	the event (trigger) on the transition
an `if` on something other than the current state (e.g. `Game.java:53`)	a guard `[condition]`
no such condition	no guard: leave it out, don't invent one
an `if` on the method's own parameter (`setAlive`)	picks the effect, not a guard
statements run with the change	the action / effect `/ action()`
early `return` when already in that state	event ignored: no transition, not a guard
a call that doesn't change the field but does something	internal transition `event / action`
an observer call (`levelWon()`)	an event sent, not a state
no way back, never used again	consider a final state ⦿

 Transition label (UML): `trigger [guard] / effect`

Write the guard in the code's terms, not prose. Two different machines:

- Level: `start() / startNPCs(); …` (no guard: the "already running" check is the event being ignored in Running, not a guard)
- Game: `start() [isAnyPlayerAlive() && remainingPellets() > 0] / …` (`Game.java:53`)

> Hint: Player.setAlive(false) has no guard. Game.start has a real guard, [isAnyPlayerAlive() && remainingPellets() > 0]. An "already running" check means the event is ignored in that state: no transition, not a guard. Leave the guard out when there is none; don't write "unconditional". A chart with no guard is fine; cite the line that shows there is no condition.

Task A › Won / Lost Are Events, Not States

What the code does (Level.updateObservers: end of every move, and in start)

- `!isAnyPlayerAlive()` → every observer gets `levelLost()`
- `remainingPellets() == 0` → every observer gets `levelWon()`
- The observer is the Game → `levelWon()` / `levelLost()` → `stop()`

There is no `won` or `lost` field anywhere.

So in your statecharts

- Level: the notification is an action (a send) on the transition / internal transition that causes it
- Game: `levelWon` / `levelLost` are events that trigger a transition
- Don't draw a "Won" or "Lost" box

Optional challenge: composite states

Use a composite (super-)state only if it simplifies the chart, and label it as an abstraction (rubric, level 4): say which field(s) it groups. Keep it inside one class's chart.

> Hint: after a win or a loss, press Start again. What does Game.start check? That answer shapes the end of your Game statechart.

Task A › PlantUML State Syntax

A starter for the handout's Level candidate (check every label):

```
@startuml SM-01
hide empty description
state "Stopped" as Stopped : inProgress == false
state "Running" as Running : inProgress == true

[*] --> Stopped
Stopped --> Running : start() / startNPCs()
Running --> Stopped : stop() / TODO
Running : move(unit, direction) / TODO
' no guard on start()/stop(): the "already
'   running" check = ignored event, not a guard
' TODO: start() does one more thing; add it
' TODO: levelWon / levelLost are events sent
'       to observers, not states
@enduml
```

Syntax

Element	PlantUML
Start/end	`[*] --> S, S --> [*]`
Transition	`A --> B : e [g] / a`
Internal	`S : e / a`
Field	`S : inProgress == true`
Choice	`state c <<choice>>`
Composite	`state Big { … }`

Write guards with the brackets (state diagrams don't add them).

(A syntax demo only; any guards, the effects and the second chart are yours to recover.)

Task A › Verify in the Debugger (Required)

Handout: "set a breakpoint in `Player.setAlive()` (when it is called with `false`) for the Player statechart, `Game.stop()` for the Game statechart, or `Level.stop()` for the Level statechart, run the game until it stops there, and include one screenshot per statechart showing the call stack and the field values."

▒▒▒ Step 1: Break on the line that assigns the field, so every caller is caught

Statechart	Breakpoint	Transition it shows
Player	`Player.java:83 this.alive = isAlive;`	Alive → Dead
Level	`Level.java:207` / `:222` (`inProgress = …`)	Stopped → Running / Running → Stopped
Game	`Game.java:54` / `:69` (`inProgress = …`)	Stopped → Running / Running → Stopped

▒▒▒ Step 2: Debug Launcher, press Start, play until a ghost catches you

- Pressing Start stops first at `Game.java:54`, then `Level.java:207` (Stopped → Running): F8 once, screenshot, Resume
- At `Player.java:83`: Frames = the stack (`setAlive` ← `playerVersusGhost` ← `playerColliding` or `ghostColliding` ← `collide` ← `Level.move` ← …) + the thread; Variables still show `alive = true`
- Step Over (F8) once → `alive = false` → screenshot now (the value after the step)
- Resume → `Game.java:69` fires via `Level.updateObservers` → `Game.levelLost` → `Game.stop` (Game chart), then `Level.java:222` via `Game.stop` → `Level.stop` (Level chart; its call stack shows `Game.stop` too, say so in the caption): F8 once at each, two more screenshots from the same run

Task A › Verify in the Debugger (cont.)

▒▒▒ Step 3: Caption every screenshot

SM-02 · Alive --setAlive(false)--> Dead · thread pool-N-thread-1 (a ghost moved onto me) · Player.java:83 SM-01 · Running --stop()--> Stopped · Level.java:222 · via Game.stop (Game.java:70)

The thread depends on who moved: AWT-EventQueue-0 if you walked into the ghost, a pool-N-thread-1 ghost task if it walked into you.

Reference screenshots for the three stops (what Frames and Variables must show): Appendix B (end of the deck).

> Hint: stop() exists on both classes: Game.stop → the Game chart, Level.stop → the Level chart. Game.stop calls Level.stop, so the Level.stop breakpoint shows Game.stop in the call stack. One screenshot per chart is the handout's requirement; screenshots of more transitions make your verification stronger.

Task B › What's in Scope Now

The whole system (A1-A2 scope + ui + sprite)

```
src/main/java/nl/tudelft/jpacman/
├── Launcher.java                     ← wires everything + maps the arrow keys
├── PacmanConfigurationException.java ← bad map / config
├── board/  game/  level/  npc/ (+ npc/ghost/)  points/   ← the core (A1-A2)
├── sprite/          ← NEW for A3: images + animation
└── ui/              ← NEW for A3: PacManUI, BoardPanel, ButtonPanel,
                       PacKeyListener, Action, ScorePanel, PacManUiBuilder
```

The real concurrency

- One `ScheduledExecutorService` per ghost (`Level.startNPCs`); `NpcMoveTask` reschedules itself (handout; `Level.java:352`)
- UI refresh: `PacManUI.start` → `nextFrame()` every 40 ms (handout's "frame timer"; `PacManUI.FRAME_INTERVAL = 40`)
- Swing Event Dispatch Thread (EDT): key presses + Start / Stop buttons, and all `paint()` calls (link 12)

> Hint: search for synchronized in level and game. Which lock protects what? That belongs in your process view. Also open PacManUI.start (its Javadoc: "the thread that redraws the interface at set intervals"): what schedules nextFrame?

Task B › Step 1: Identify the Style

▒▒ Candidates (Architectural Design slides): pick one or a combination, and justify it from imports

Style	What to look for in JPacman
Layered	higher packages use lower ones only? closed or open? any arrow up / sideways?
MVC	who plays Model, View, Controller? how does the view learn of a change?
Pipe-and-filter	a stream through independent filters? argue for or against

▒▒ Then: where does the code differ from the textbook?

- The handout's example: there is no explicit controller package. The controller role lives inside `ui` (`PacKeyListener`, `ButtonPanel`, the `Action` interface), with the key `Action`s created in `Launcher` and Start / Stop as `game::start` / `game::stop` in `PacManUiBuilder`: view and controller are not separated (Architectural Design slides: "Division between the View and the Controller is not very clear in some cases")
- Which packages import `sprite`? The model classes reference `Sprite` (abstract `Unit.getSprite()`, `Player.deathSprite`, `Ghost.sprites`) and `Sprite.draw` takes a `java.awt.Graphics`. "Core never imports `ui`" is true and not enough: decide whether `sprite` is view or a lower layer, and justify it
- Is there a cycle between packages?
- Does the model notify the view, or does the view ask? (Who implements `LevelObserver`?)

> Note: "It's MVC" with no evidence scores low. Every claim → a package dependency or class + method.

Task B › Step 2: The Five 4+1 Views

From the 4+1 slides (Which Diagram for Which View?), mapped to JPacman

View	Question it answers	UML diagram	Yours
Logical	main parts, how related?	class / package diagram	LV-01
Development	how is the code organised? who imports whom?	package diagram + imports	DV-01
Process	what runs at the same time? how do they talk?	sequence / activity diagram	PV-01
Physical	where does it run?	deployment diagram	PHY-01
+1 Scenario	one use case through all the views	sequence diagram (+ use case)	SC-01, SC-02

Each view: a diagram + a short explanation (handout). Physical: "a short description is enough".

Logical vs. development

The handout accepts a package diagram for the logical view and lets you reuse it for the development view. The 4+1 slides draw the logical view as a class diagram (ATM) and the development view as packages + imports: "Same classes as the logical view, grouped the way programmers work."

> Tip: a class-level LV-01 and a package-level DV-01 show more.

Task B › Logical View (LV-01) and Physical View (PHY-01)

LV-01: the key abstractions

"This is the class diagram you already know, seen as architecture." (4+1 slides)

- Start from CD-01, add the ui classes (PacManUI, BoardPanel, ScorePanel, ButtonPanel, PacKeyListener, Action) and the sprite types the model references
- Cut it down: only architecturally significant classes and relationships (Level → Game through Level.LevelObserver; Level → Board, CollisionMap; Game and PlayerCollisions → PointCalculator; the Unit and Ghost hierarchies; BoardPanel → Game)
- No attribute lists; operations only where they carry the architecture

PHY-01: one node

"A simple program may have only one node … That is still a physical view." (4+1 slides)

```
@startuml PHY-01
node "Student laptop" {
  node "JVM 11" {
    artifact "jpacman (Launcher)"
  }
}
@enduml
```

One sentence is enough: one desktop, one JVM, no network. Say what the ghost threads and the EDT share: one process, one heap.

Task B › Development View (DV-01): Package Diagram

```
@startuml DV-01
' keep "npc.ghost" as one package name
set separator none
package "nl.tudelft.jpacman" {
  package ui {
  }
  package game {
  }
  package npc.ghost {
  }
  ' TODO: board, level, npc, points, sprite
}
ui ..> game : <<import>>
' TODO: one arrow per real import dependency
@enduml
```

Rules

- One package per Java package (+ Launcher, the root class)
- One dashed arrow per real import relationship, pointing to the package used
- Write the import rules in words (4+1 slides: "bank.model never imports atm.ui")
- Report cycles: allowed inside one layer, a deviation across layers
- Layers / M-V-C? A labelled rectangle { … } + why

Read the imports, don't guess (run in src/main/java/nl/tudelft/jpacman)

```
# macOS: the jpacman imports of ONE package (repeat: level, npc, npc/ghost, …)
grep -h "^import nl.tudelft.jpacman" board/*.java | sort -u
# who imports ui? expected: Launcher.java (+ ui's own ScoreFormatter imports); no core package
grep -rn "import nl.tudelft.jpacman.ui" .

# Windows (cmd)
findstr /c:"import nl.tudelft.jpacman" board\*.java
findstr /s /n /c:"import nl.tudelft.jpacman.ui" *.java
```

Task B › Process View (PV-01)

```
@startuml PV-01
skinparam componentStyle rectangle
node "One JVM process" {
  rectangle "Swing EDT" <<thread>> as edt
  rectangle "Ghost task (x N)" <<thread>> as ghosts
  rectangle "UI refresh (40 ms)" <<thread>> as frame
  rectangle "level : Level" as level
}
edt --> level : TODO (via Game.move)
ghosts --> level : TODO
' TODO: what does the UI refresh read?
' TODO: which lock serialises the moves?
' TODO: 4 ghosts on board.txt; say so
@enduml
```

(A syntax demo only; the arrows and labels must come from the code.)

What the view must show

- Every thread of control
- What starts / stops each one (e.g., ghost tasks: `Level.start` / `stop`)
- Shared state + the lock that guards it
- Sync vs. async communication (4+1 slides: dashed = reply that comes back later)

Other options: a sequence diagram with one lifeline per thread, or an activity diagram with `fork` (link 11).

Task B › +1 Scenario (SC-01 · SC-02): One Key Press Through the Views

Two flows, not one call chain (handout)

SC-01: input → model update (EDT)

```
@startuml SC-01
actor Player as user
participant ": PacKeyListener" as keys
participant "game:SinglePlayerGame" as game
participant "level:Level" as level

user -> keys ++ : keyPressed(event)
' TODO: Action.doAction() -> Game.move
'       -> Level.move -> ...
@enduml
```

Follow it to where the state changes (score, square, alive) and the observers run.

SC-02: the UI's own refresh

```
@startuml SC-02
participant "scheduler" as T
participant ": PacManUI" as ui
participant ": BoardPanel" as panel

loop every 40 ms (FRAME_INTERVAL)
  T -> ui ++ : nextFrame()
  ui -> panel : repaint()
  ' TODO: what else does nextFrame refresh?
  deactivate ui
end
...later, on the Swing EDT...
[-> panel ++ : paint(g)
' TODO: what does paint() read?
deactivate panel
@enduml
```

[-> panel: the sender is outside the diagram (Swing's EDT).

> Hint: repaint() only requests a repaint and returns; Swing calls paint() later on the EDT (link 12), so never draw paint nested inside repaint.

Task B › +1 Scenario: The Table That Makes It the "+1"

One key press, traced through the views (4+1 slides: Step · Where it shows up)

Step	Where it shows up
1. arrow key → keyPressed → doAction	Process: Swing EDT · Development: Launcher → ui, game
2. Game.move → Level.move → collide	Logical: Game, Level, Square, PlayerCollisions
3. later: nextFrame → repaint(); Swing → paint	Process: refresh thread → EDT · Physical: 1 JVM

Draw the two flows as separate sequence diagrams (SC-01, SC-02); a table like this helps show the four views fit together (not required). In PlantUML, `[-> x : m()` draws a message from a sender outside the diagram; use it whenever the caller isn't a lifeline (Swing, a timer).

How do the two flows connect?

- Once SC-01 enters game, nothing calls back into ui (it does touch sprite: see Player.setAlive)
- SC-02 never receives a message from the model: it reads shared state (Board, Square, Unit) on its own timer
- So explain in words how the change made in SC-01 becomes visible in SC-02: shared state the view reads, not a message

> Hint: that polling, and the fact that paint reads the board without Level.moveLock (BoardPanel's only field is game; the lock is private to Level), is also process-view material (PV-01).

Task C › What an ADR Is

One short note = one significant design decision

The handout requires five parts: context · decision · alternatives · consequences · code evidence. The template below adds a title, a status and a Related line (extra reference link 7 has the original format):

```
ADR-01: <short noun phrase, e.g. "Abstract Ghost with per-ghost AI move">
Context:      the forces / problem the code had to handle (facts, neutral)
Decision:     what the code does, in full sentences ("The design uses ...")
Alternatives: ≥ 1 reasonable other design, and why it is not the one used
Consequences: what it BUYS and what it COSTS, with a concrete example
Evidence:     class.method (file:line), Find Usages / grep results
Status:       Accepted (implemented in the code)          ← optional
Related:      requirement keys (SJC-…), diagram IDs (SM-01, …) ← optional
```

Consequences = the trade-off argument

Not "it's decoupled, that's good". Buys: a change it makes easy (Modular Design: OCP, DIP, low coupling). Costs: indirection, a runtime failure instead of a compile error, performance, a harder trace. One concrete example from this code.

What counts as "significant"?

Affects structure, non-functional characteristics, dependencies, interfaces, or construction techniques (extra reference link 7); concurrency choices count as structure. Name the design idea in plain words (for example, 'Level tells Game when the level is won'). You do not need a pattern name.

> Note: analyse the design as it is. Don't claim what the developers "considered"; propose the alternatives yourselves.

Task C › The Handout's Example, Started

```
ADR-01: Abstract Ghost with a per-ghost AI move

Context:  Four ghosts share movement rules
          (squares, intervals, sprites) but
          chase differently.
Decision: Ghost (npc) is abstract; nextMove()
          calls the abstract nextAiMove() and
          falls back to randomMove(). Blinky /
          Pinky / Inky / Clyde each implement
          nextAiMove(). Moving itself is
          Level.move, not a Ghost method.
Alternatives: TODO (e.g. one Ghost class + switch on
          GhostColor? a separate class per ghost
          behaviour?)
Consequences: TODO (+ adding a ghost = new subclass;
          − ...?)
Evidence: Ghost.nextMove(), Ghost.nextAiMove(),
          Blinky.nextAiMove() ... TODO file:line,
          Find Usages of nextAiMove
```

▒▒▒ Then find ≥ 1 more yourself

Places to look (find the evidence):

- `Level.LevelObserver` (Level reports events to `Game` through an interface)
- `PointCalculator` + `PointCalculatorLoader` + `scorecalc.properties`
- `CollisionMap` and its three implementations (which one runs?)
- `ui.Action` (buttons / keys vs. what they do)
- The UI polling the model on a timer instead of being notified

> Hint: the handout's "per-ghost move method" is nextAiMove(); there is no move on Ghost. Two ADRs is the minimum; this can be one of them.

Task C › Extend the RTM in Requirement Yogi

Same mechanism as A2 (A2 slides › Task D › Start the RTM)

- Each A3 diagram / ADR sits in its own section with its ID (SM-01, LV-01, PV-01, ADR-01 …)
- In each section, reference the requirement keys it covers (e.g., SJC-001) → Yogi records where each one is cited
- Link an ADR only where the relationship is real (not every requirement traces to an ADR)

What a row should end up showing

Requirement	A2 design	A3 statechart	A3 views	ADR
SJC-002 Player dies on ghost	CD-01, SD-0x	SM-0x (Player)	PV-01, SC-01	— or ADR-0x
SJC-003 No player alive → lost	SD-0x (alt)	SM-0x (Level, Game)	PV-01	ADR-0x (Level → Game notification)
SJC-004 Last pellet → won	CD-01	SM-0x (Level, Game)	LV-01	ADR-0x (Level → Game notification)

One row per modelled requirement; your keys and IDs may differ.

Export

Requirement Yogi → Traceability (Yogi Cloud: Requirements in the space sidebar) → add the Links column → Export to Excel or screenshot → Traceability section of the architecture page.

> Careful: Yogi's Links column points to the page where a key is quoted. With everything on one page, every row shows the same page. Either give each diagram / ADR its own child page, or add the manual table above next to the export (a manual "requirement → realised by" table is accepted). Every modelled requirement must appear.

The Architecture Page in Confluence

Confluence → Create → Blank page, e.g., "Architecture — JPacman"

Sections

1. Statecharts: SM-01 … SM-03, each with its code citations and its debugger screenshot
2. Architectural style + justification (imports, deviations from the textbook)
3. 4+1 views: LV-01 · DV-01 · PV-01 · PHY-01 · SC-01 + SC-02, each with a short explanation
4. ADRs: ADR-01, ADR-02, …
5. Traceability: Yogi export (+ manual table)
6. Team & process: roles, retro (keep / change / try), AI Usage note

Embedding diagrams

- Type `/image` → upload the rendered PNG (screenshots the same way)
- One image per diagram, captioned with its ID
- Diagrams must be rendered: no raw PlantUML text instead of a picture

AI Usage note

"Any use of any outside source or AI tool must be documented" (handout). Do it here: whether and how AI tools were used, and for what. The repo's `AI_USAGE.md` template is a good structure to copy.

Export

Page → ⋯ → Export → PDF → rename to architecture.pdf

Self-Check Before You Submit

- ☐ ≥ 2 statecharts: states, events, actions, and guards only where the code has a condition (none invented); every state → a field / condition / event; won / lost as events
- ☐ Statecharts and screenshots: one debugger screenshot per statechart (call stack + field value after the step), captioned with the transition
- ☐ Style (or combination) identified and justified from package dependencies; differences from the textbook discussed (`sprite`; cycles; controller role inside `ui` + `Launcher`, no controller package)
- ☐ All five views: LV-01 (logical) · DV-01 (development) · PV-01 (process: real threads + locks) · PHY-01 (physical) · SC-01 + SC-02 (scenario: input flow separate from UI refresh)
- ☐ ≥ 2 ADRs: context, decision, alternatives, consequences (buys and costs), code evidence
- ☐ RTM extended (Traceability section): every modelled requirement → relevant statecharts and views, and an ADR where meaningful
- ☐ uml-sources.zip has every `.puml`; Team & process section (roles, retro, AI Usage note)
- ☐ Jira: epic, assigned tasks, stand-up comments, roles rotated; instructor (kahani@sce.carleton.ca) + all TAs added; the link opens
- ☐ Submitted on Brightspace: architecture.pdf + uml-sources.zip + board link; all names + student IDs

> Remember: "Work that is not clearly explained or not traceable to the actual code will not be considered correct." Any team member may be asked to present it.